

Hybrid Static-Trace Analysis for Synthesizing Interaction Diagrams in Event-Driven Systems

Scenario-Specific Interaction Diagrams in SysML v2

Student:	Prathik Anand Krishnan
Daily Supervisor:	Dr. Rosilde Corvino, TNO – ESI
Academic Supervisor:	Dr. L. Thomas van Binsbergen (CCI, UvA)
Second Reader:	Prof. Dr. Benny Åkesson (PCS, UvA)
Host Organization:	TNO – ESI, Eindhoven, High Tech Campus 25



Agenda

- 1 Motivation
- 2 State-of-the-art & Gaps
- 3 Research Questions
- 4 Approach, Methodology & Implementation
- 5 Experiments
- 6 Results & Findings
- 7 Contributions, Limitations & Conclusion

Problem Statement & Motivation

Large Event-Driven Systems



Traces - Incomplete / Noisy - often instrumentation limited



Static Code Analysis - Over-Approximates behaviour



Reverse-engineered manually & Costly to maintain



No Scenario-Specific Accurate Interaction Diagrams

State of the Art & Research Gap

Work	Static	Dynamic	Hybrid	Scenario-Scoped	Interconnection view	Sequence view	UML	SysML v2
Grove et al. (1997) [1]	✓				✓			
Briand, Labiche & Leduc (2006) [2]		✓		✓		✓	✓	
Labiche, Kolbah & Mehrfard (2013) [3]	✓	✓	✓	✓		✓	✓	
Abdelmalek, Khriss & Jakimi (2023) [4]	✓	✓	✓			✓	✓	
This Thesis	✓	✓	✓	✓	✓	✓		✓

Research Gap:

No existing approach combines static structure and runtime traces to infer interactions for scenario-scoped reconstruction of SysML v2 interconnection and sequence models.

Research Questions

Hybrid Analysis - Interaction Diagram

Static Code
Structure

Runtime Traces

RQ1: How does combining static analysis and runtime traces improve accuracy and completeness of scenario-specific interaction model reconstruction?

RQ2: How does guiding trace collection with the static call graph remove noise without losing scenario coverage?

RQ3: How well do reconstructed SysML v2 interaction (interconnection and sequence) diagrams consolidate static and dynamic evidence into abstract artifacts sufficient for system behavior comprehension?

Codebase Selected: CpsCore [7]

Codebase overview:

- A Modular Autopilot Framework for UAVs [9]
- 154 source/header files
- ~13,000 LOC (3,800 in .cpp files, 9,200 in headers)
- Heavy template-based static polymorphism (~470 occurrences)
- No cross-module call resolves through a virtual table. Inter-module calls go through concrete class references or *boost::signals2* (external dependency) slot connections.
- Test suite: 1,425 assertions across 61 test cases is the ground truth for behavioural verification.

Approach: Agent Skill Pipeline

Skill	Role in Final Pipeline
get-neo4j-schema	Static Analysis - graph introspection
text-to-cypher + get-cypher + cypher-to-output	Scenario-scoped query generation, execution & standalone Cypher authoring
get-interface-dependency-table	Static Analysis - Structural dependency extraction
get-sequence-dependency-table	Static Analysis - Sequence / call chain extraction
draw-sysml-structural-model	SysML v2 Output - BDD/IBD
draw-sysml-sequence-model	SysML v2 Output - Sequence Diagram
add-runtime-instrumentation	Trace Collection - csp_matcher probe injection
agentic-synthesis	Evidence judgement, pruning false positives
draw-mermaid-structural-diagram	Mermaid Diagrams - For Quick Analysis
draw-mermaid-sequence-diagram	Mermaid Diagrams - For Quick Analysis

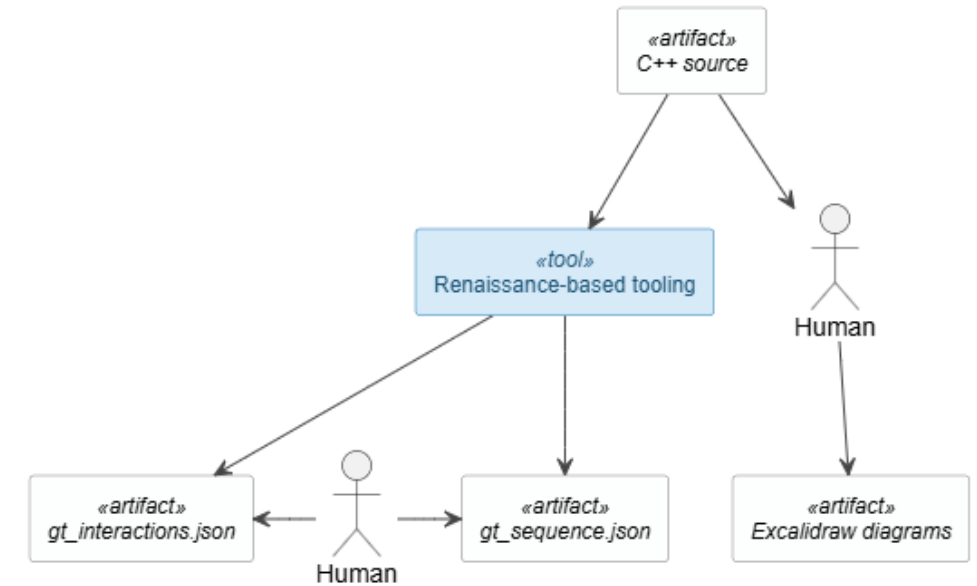
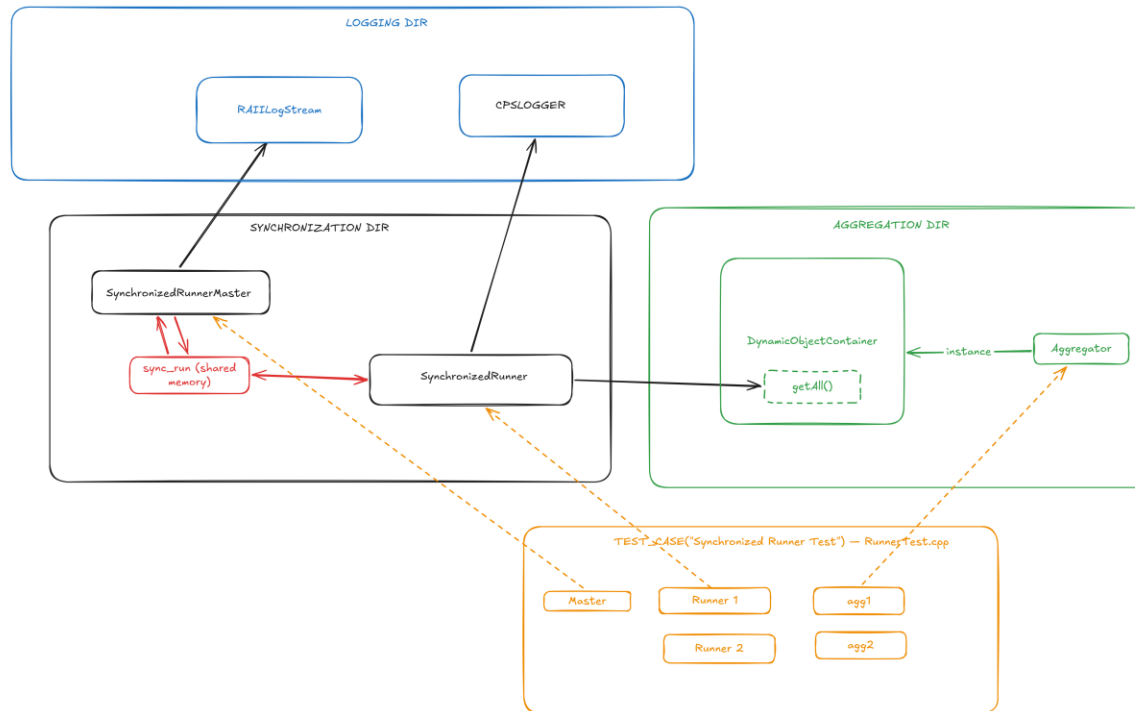
- Twelve modular, composable, LLM-invokable skills: eleven follow a fixed rule set, one (agentic synthesis) applies judgement instead.
- Key design property: Each skill is independently testable. Scenario focus is expressed as a natural-language query, translated into a Cypher query that scopes the graph before instrumentation runs — this is what makes guided instrumentation possible, the mechanism RQ2 actually tests (guiding trace collection with the static graph to remove noise without losing coverage).

Four Evaluation Scenarios were Selected

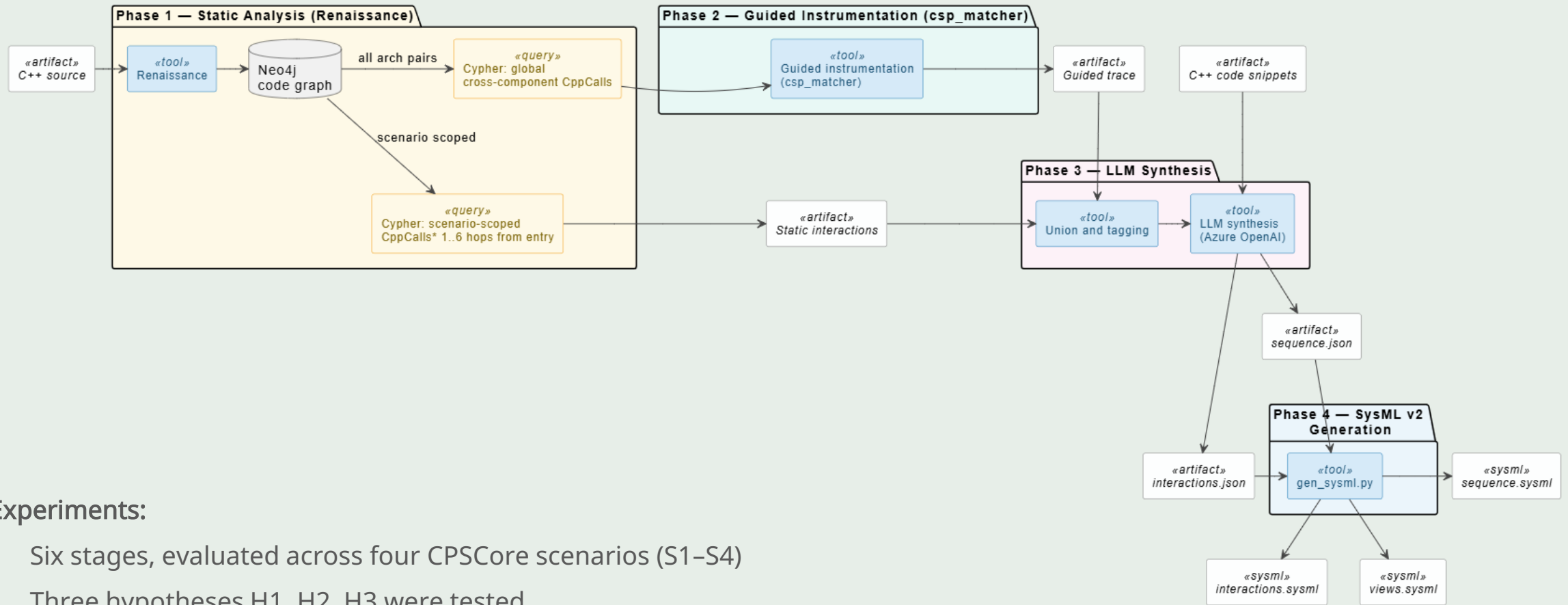
Scenario	Nature	What it exercises	Test Case Reference
S1	Pre-existing	Aggregation chain	RunnerTest.cpp - "Synchronized Runner Test"
S2	Pre-existing	Config mapping	PropertyMapperTest.cpp - "Optional test"
S3	Pre-existing	Multi-component runner	SchedulerTest.cpp - "MultiThreaded Test 1"
S4	Constructed	Publish/subscribe	StageEventBridgeTest.cpp - "Stage Event Bridge Pub-Sub Routing"

Ground Truth

S1 — Synchronized Runner & Aggregator Notification Chain (Ground Truth)



Methodology - The Hybrid Pipeline



Experiments:

- Six stages, evaluated across four CPSCore scenarios (S1–S4)
- Three hypotheses H1, H2, H3 were tested.

H1 Experiment Design

These experiments were run with **full and slice traces**, both computed via clang-renaissance

Condition	Phase 1 (Renaissance/Neo4j)	Phase 2 (Guided Instrumentation)	Phase 3 (Synthesis)	Phase 4 (SysML)
C1 LLM-only	X	X	LLM + source only (no static/dynamic evidence)	X
C2 Static-only	✓ full	X	X (Static graph only)	X
C3 Dynamic-only	X	✓ full	X (de-duplicated + ordered trace)	X
C4 Static + Dynamic	✓	✓	Set union (no LLM)	X
C5 Intersection	✓	✓	Set intersection (no LLM)	X
C6 Full pipeline	✓	✓	Full LLM synthesis (C2+C3+source)	✓

H1 – Metrics Calculations Example

S1- Reference (REF):

```
[ { "source": "Synchronization", "target": "Aggregation", "interaction": "Aggregator.getAll" }, // Runner.Test
  { "source": "Synchronization", "target": "Logging", "interaction": "CPSLogger.flush" }, // Runner.Test
  { "source": "Synchronization", "target": "Aggregation", "interaction": "Aggregator.getAll" }, // Runner.Timeout
  { "source": "Synchronization", "target": "Logging", "interaction": "CPSLogger.flush" }, // Runner.Timeout
  { "source": "Synchronization", "target": "Logging", "interaction": "RAIILogStream.stream" } ] // Runner.Timeout
```

S1- C1_LLM_only (GEN):

```
[ { "source": "Synchronization", "target": "Aggregation", "interaction": "Aggregator.getAll" }, TP1 // Runner.Test
  { "source": "Synchronization", "target": "Logging", "interaction": "CPSLogger.flush" }, TP2 // Runner.Test
  { "source": "Synchronization", "target": "Logging", "interaction": "RAIILogStream.stream" } ] TP3 // Runner.Timeout
[ { "source": "Synchronization", "target": "Aggregation", "interaction": "Aggregator.getAll" }, FN1 // Runner.Timeout
  { "source": "Synchronization", "target": "Logging", "interaction": "CPSLogger.flush" } ] FN2 // Runner.Timeout
```

TP=3, FP=0, FN=2
 $P=3/(3+0)=1.00$
 $R=3/(3+2)=.60$
 $F1=2(1)(.6)/(1.6)=.75$

So, TP=3 and FN=2: C1 gets both Runner.Test edges right but on Runner.Timeout reports only stream, missing getAll/flush for that test entirely. An LLM reasons over the raw scenario description alone over the source, no static graph, no trace is provided. Baseline for what's inferable with zero pipeline evidence.

H1 Results: Interconnections and sequence F1

Interactions F1 — all scenarios

Scenario	C1	C2	C3	C4	C5	C6
S1	0.750	0.762	0.829	0.661	1.000	1.000
S2	0.000	1.000	1.000	1.000	1.000	1.000
S3	0.333	0.500	1.000	1.000	0.500	0.857
S4	0.400	0.667	0.667	0.500	1.000	0.500
Macro F1	0.371	0.732	0.874	0.790	0.875	0.840

Sequence F1 — all scenarios

Scenario	C1	C2	C3	C4	C5	C6
S1	0.667	0.306	0.758	0.758	1.000	1.000
S2	0.000	1.000	1.000	1.000	1.000	1.000
S3	0.560	0.143	0.867	0.867	0.267	0.896
S4	0.000	0.667	0.667	0.667	1.000	0.667
Macro F1	0.307	0.529	0.823	0.823	0.817	0.891

Macro F1 with guided vs full instrumentation

	Intr (guided)	Intr (full)	Seq (guided)	Seq (full)
C3 / C3f	0.865	0.831	0.810	0.771
C4 / C4f	0.764	0.764	0.810	0.771
C5 / C5f	0.900	0.833	0.853	0.787
C6 / C6f	0.871	0.900	0.913	0.873

H1 Interpretation — Why the Numbers Look This Way

- **C5 (C2 $\cap$ C3 intersection) is the best condition for interactions (macro=0.875).**
In every scenario where both static and dynamic capture the same edge, the intersection removes noise from both sources.
- **C6 (LLM + static + dynamic + source code) is the best condition for sequence (macro=0.891).** Adding source code enables the LLM to identify and remove test-fixture setup events from the sequence output.
- **Guided $\geq$ Full** for C3-C6 interconnections and sequence.

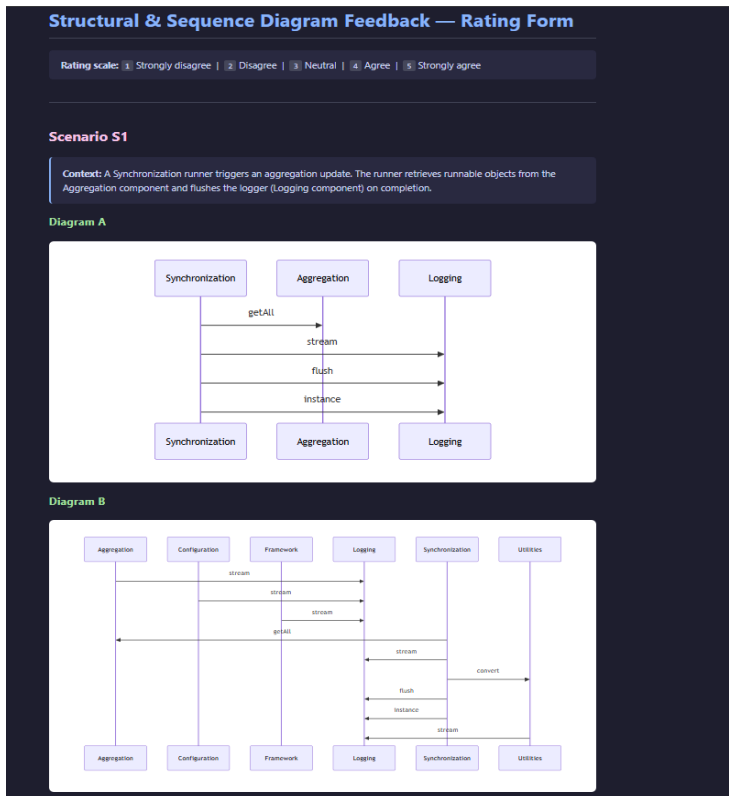
H2 Experiment

Setup:

A blind A/B comparison rating form was used:

For each scenario (S1–S3), a rater scores two anonymized diagram variants (A vs. B, unlabelled generation method) on 3 questions relevance to the scenario, readability, and practical usefulness on a 1–5 scale plus free-text comments.

It closes with two overall questions on which style was preferred and what features they'd want added.



H2 Results in Detail: Noise, Coverage & Expert Ratings

- **Setup:** Full instrumentation (probe everything, filter after) vs. Guided (slice first, probe only scenario-relevant call sites), on S1-S3.
- **Standout:** Across 6 raters × 9 scenario-dimension pairs = 54 comparisons: **Guided rated strictly higher in 47**, tied in 6, lower in exactly 1: The single dissent traced to collapsing 7 repeated Configuration→Logging into 1. A trade-off, not a defect.

Metric	Full	Guided	What it means
Nodes (mean)	6.0	3.0	Full pulls in 2× as many components per scenario as guided on average.
Edges (mean)	9.0	2.7	Full shows 3.3× as many component-pairs on average.
Coverage (0-1.0)	1.000	1.000	Identical
Noise (0-1.0)	0.704	0.000	70% of Full doesn't belong to scenario.
Accuracy (6-rater mean, 1-5)	2.39	4.61	Raters penalize diagrams where they can't tell which edges actually matter.
Readability (6-rater mean, 1-5)	2.44	4.89	Fewer nodes/edges directly means less to visually parse.
Usefulness (6-rater mean, 1-5)	2.94	4.00	One rater found a few "extra" Full-edges is marginally useful as context even while rating them as noise elsewhere.

H3 Setup & Results : Diagram Utility

- **Setup:** Does the pipeline's output (the C5 interaction list) help an LLM agent comprehend a scenario — alone, and in combination with source and traces? A single holistic experiment, five conditions, across all four scenarios.
- **Standout:** Diagram only equals Source+Trace+Diagram on every measure. Even with no source and no trace, the reconstructed diagram alone is already a sufficient basis for scenario comprehension.

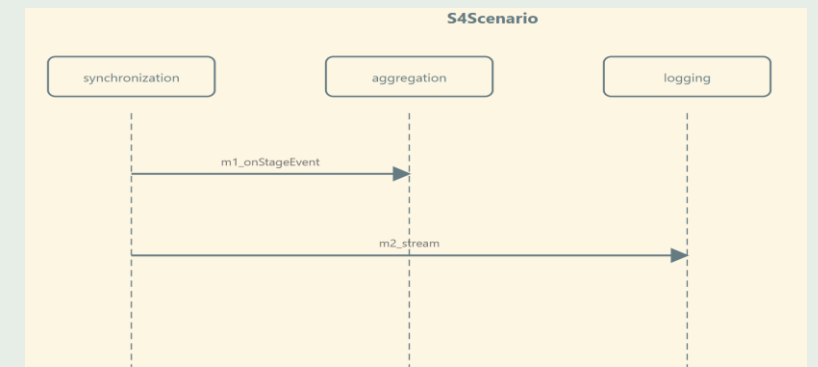
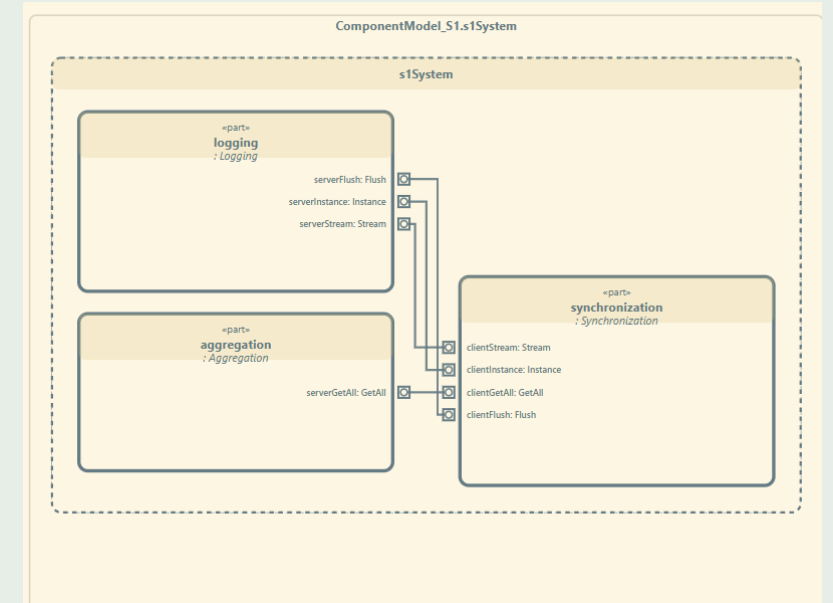
Condition	Component recall	Interaction recall	Completeness (1–5)
Source only	0.917	0.800	4.25
Trace only	0.917	0.900	4.25
Diagram only	1.000	1.000	5.00
Source + Trace	0.917	0.900	4.25
Source + Trace + Diagram	1.000	1.000	5.00

What Was Produced: SysML v2 Artifacts

Structural, Sequence and Interconnection Model:

- SysML v2 structural and sequence models are generated using the **draw-sysml-structural-model** and **draw-sysml-sequence-model** agent skills.
- Each element becomes a typed port pair (calls<X> / provides<X>) joined by a connect statement in the structural model, representing provider-consumer relationships between components.
- Each sequence step becomes a labelled SynchronousCall part (with ref from/ref to Lifeline references) in the sequence model.
- Interaction flows built from interactions.json / sequence.json.

SysML v2 models are generated from C6 (the full pipeline) output, one structural, one sequence diagram per scenario (S1, S2, S3, S4)



Practitioner Validation

Goal:

Does the Static Half Transfer to a Second, Larger Codebase?

Setup:

Renaissance MCP: The static-extraction + graph-querying backend exposed as an MCP tool inside GitHub Copilot.

Trialled by 4 engineers at an industrial partner on a **second C++ codebase, orders of magnitude larger than CPSCore**

Not a repeat of H1/H2/H3: unblinded, semi-structured hands-on session + interviews with static-extraction only, no runtime tracing.

Interview Findings	Thesis Connection
P3 resolved the proxy-leak defect using only the static graph in 5 min vs. ~5 months (anecdotal).	Same mechanism as S4/C2 : static extraction alone finds architecturally-present, runtime-invisible interactions.
P2 's tool found a stub definition invisible to code search.	Static catches blind spots a disagreeing dynamic source could confirm. But untested here (no trace was collected).
Codebase leans heavily on proxies, virtual interfaces, dynamic connect/disconnect.	Exactly the regime where C5 should beat C4 , per the S4 mechanism. Predicted, not measured, at this scale. Future Work
Dependency/structural diagrams rated useful by all 4, cleaner than existing tooling for P3.	Validates the interface dependency table. This thesis's one of the core static artifact.
Sequence diagrams valued, but event labels judged too abstract without inline detail.	Motivates the hover-detail request via the existing sourceLineNumber property carried on CppCalls edges — future work for SysML generation.

Research Questions: Where Each Stands

RQ	Question	Status
RQ1	Does combining a static call graph with runtime execution traces reconstruct scenario-specific Interaction Models more accurately than either source alone?	Confirmed (H1): Intersection (C5) reaches macro-F1 = 0.875. Full pipeline (C6) reaches 0.840. Sequence (C6) reaches macro-F1 = 0.891. Guided trace $\geq$ Full trace for C3-C6 interconnections and sequence.
RQ2	Does guiding trace collection with the static call graph remove reconstruction noise without losing scenario coverage?	Confirmed (H2): Guided instrumentation cuts retained nodes by 50% and edges by 70%; A blinded panel preferred guided diagram in 47 of 54 comparisons.
RQ3	How well does a reconstructed SysML Interaction Model consolidate static and dynamic evidence into a single artefact sufficient for system-behaviour comprehension?	Confirmed (H3): The diagram alone allows LLM alone reaches full component recall, interaction recall and completeness.

Challenges Encountered & How They Were Resolved

Challenges	Solutions
C++ is context-sensitive — regex/simple parsers fail	Used the Renaissance semantic-extraction tool + Neo4j property graph.
Thousands of dependencies make SysML toolchains very slow	scenario-scoped Cypher queries trim the graph before generation
Hardcoded scenario scoping is brittle	Agent skill architecture (12 skills) expresses scenario definition as a natural-language query via text-to-cypher
Traces alone even sliced contains noise (setup-teardown).	Scenario-scoping helps with improving the precision.

Overall Summary

What was proposed:

Hybrid static + trace pipeline → scenario-specific SysML v2 interaction diagrams

What was delivered:

- Neo4j knowledge graph of CPSCore - 121 components, 2,574 ordered static interactions
- 12 composable agent skills spanning extraction → scenario scoping → instrumentation → agentic synthesis → SysML v2 diagram generation
- csp_matcher runtime instrumentation tool.
- H1/H2/H3 confirmed on 6 scenarios: Intersection macro-F1 0.875 and Full pipeline macro-F1 0.840; guided instrumentation cuts noise 100% with zero coverage loss; the reconstructed diagram alone achieves perfect comprehension recall

Beyond the plan:

Exploratory practitioner session - 4 engineers, second much larger industrial codebase, via GitHub Copilot MCP integration; resolved a real proxy-leak defect using only the static graph.

Future work:

Controlled replication on a second system, richer diagram labelling & interactivity and improved pipeline.

References

- [1] Grove, D., DeFouw, G., Dean, J., Chambers, C. "Call Graph Construction in Object-Oriented Languages." Proc. 12th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), 108–124, 1997.
- [2] Briand, L. C., Labiche, Y., Leduc, J. "Toward the Reverse Engineering of UML Sequence Diagrams for Distributed Java Software." IEEE Transactions on Software Engineering, 32(9), 642–663, 2006.
- [3] Labiche, Y., Kolbah, B., Mehrfard, H. "Combining Static and Dynamic Analyses to Reverse-Engineer Scenario Diagrams." Proc. 29th IEEE International Conference on Software Maintenance (ICSM), 130–139, 2013.
- [4] Abdelmalek, H., Khriss, I., Jakimi, A. "Towards a Hybrid Approach to Reverse Engineer Behavioral UML Diagrams from Source Code." Journal of Theoretical and Applied Information Technology (JATIT), Vol. 101, No. 22, 30 November 2023.
- [5] Kienle, H. M., Müller, H. A. "Requirements of Architecture Reconstruction Tools." Architectures for Adaptive Software Systems (LNCS), 1–22, 2009.
- [6] Manning, C. D., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [7] Theile, M., Dantsker, O. D., Nai, R., Caccamo, M., Yu, S. "uavAP: A Modular Autopilot Framework for UAVs." *AIAA AVIATION 2020 Forum*, 3268, 2020. (*CPSCore, the case study framework*)